Simulation and Performance Evaluation of Cross-Platform Edge AI Model Deployment using ONNX and PyTorch Mobile

Objective:

To compare the deployment of an edge AI model on two cross-platform runtimes, PyTorch Mobile and ONNX Runtime, in terms of inference latency, model size, memory usage and accuracy, using simulated performance data, and to calculate the improvement of one over the other.

Procedure:

1. Define the simulated performance data for the two deployment platforms: inference latency (24.8 ms for PyTorch Mobile, 18.6 ms for ONNX Runtime), model size (12.4 MB, 10.8 MB), memory usage (86.0 MB, 72.0 MB) and accuracy (94.2%, 94.0%).
2. Store the data in a table and print the performance evaluation.
3. Calculate the percentage reduction of ONNX Runtime relative to PyTorch Mobile for latency, model size and memory, and the accuracy difference.
4. Plot four bar graphs (latency, model size, memory usage and accuracy) with the value labelled on each bar.
5. Save the graph as an image and the results as a CSV file.

Program:

```
# Install required libraries
!pip -q install pandas matplotlib

import pandas as pd
import matplotlib.pyplot as plt
from google.colab import files

# ========================================
# 1. SIMULATED PERFORMANCE DATA
# ========================================

platforms = ["PyTorch Mobile", "ONNX Runtime"]

data = {
    "Inference Latency (ms)": [24.8, 18.6],
    "Model Size (MB)": [12.4, 10.8],
    "Memory Usage (MB)": [86.0, 72.0],
```

```
    "Accuracy (%)": [94.2, 94.0]
}

df = pd.DataFrame(data, index=platforms)
df.index.name = "Deployment Platform"

print("=" * 55)
print("EDGE AI DEPLOYMENT PERFORMANCE EVALUATION")
print("=" * 55)
print(df.round(2))

# ========================================
# 2. CALCULATE PERFORMANCE IMPROVEMENT
# ========================================

latency_improvement = (
    (24.8 - 18.6) / 24.8
) * 100

size_improvement = (
    (12.4 - 10.8) / 12.4
) * 100

memory_improvement = (
    (86.0 - 72.0) / 86.0
) * 100

print("\nPERFORMANCE COMPARISON")
print(f"Latency reduction: {latency_improvement:.2f}%")
print(f"Model size reduction: {size_improvement:.2f}%")
print(f"Memory reduction: {memory_improvement:.2f}%")
print("Accuracy difference: 0.20 percentage points")

# ========================================
# 3. PLOT FOUR COMPARISON GRAPHS
# ========================================

fig, axes = plt.subplots(2, 2, figsize=(13, 9))

metrics = [
    ("Inference Latency (ms)", "Inference Latency", "Time (ms)"),
    ("Model Size (MB)", "Model Size", "Size (MB)"),
    ("Memory Usage (MB)", "Memory Usage", "Memory (MB)"),
    ("Accuracy (%)", "Model Accuracy", "Accuracy (%)")
]

for ax, (column, title, ylabel) in zip(axes.flat, metrics):
    values = df[column].values

    bars = ax.bar(platforms, values)
    ax.set_title(title)
    ax.set_ylabel(ylabel)
    ax.grid(axis="y", alpha=0.3)
    ax.tick_params(axis="x", rotation=10)

    for bar, value in zip(bars, values):
        ax.text(
            bar.get_x() + bar.get_width() / 2,
            bar.get_height(),
            f"{value:.1f}",
```

```
            ha="center",
            va="bottom"
        )

plt.suptitle(
    "Cross-Platform Edge AI Deployment: ONNX vs PyTorch Mobile",
    fontsize=14
)
plt.tight_layout()

# Save graph image
plt.savefig(
    "Edge_AI_Performance_Output.png",
    dpi=180,
    bbox_inches="tight"
)
plt.show()

# ========================================
# 4. SAVE RESULTS
# ========================================

df.to_csv("Edge_AI_Performance_Results.csv")

print("\nSimulation completed successfully.")
print("Output image: Edge_AI_Performance_Output.png")
print("Results file: Edge_AI_Performance_Results.csv")

# Download both files from Colab
files.download("Edge_AI_Performance_Output.png")
files.download("Edge_AI_Performance_Results.csv")
```

Output:

```
======================================================
EDGE AI DEPLOYMENT PERFORMANCE EVALUATION
======================================================
                    Inference Latency (ms)  Model Size (MB)  Memory Usage (MB)  Accuracy (%)
Deployment Platform
PyTorch Mobile                        24.8             12.4               86.0          94.2
ONNX Runtime                          18.6             10.8               72.0          94.0

PERFORMANCE COMPARISON
Latency reduction: 25.00%
Model size reduction: 12.90%
Memory reduction: 16.28%
Accuracy difference: 0.20 percentage points

Simulation completed successfully.
Output image: Edge_AI_Performance_Output.png
Results file: Edge_AI_Performance_Results.csv
```

Fig. 1: Console output (performance table and comparison)

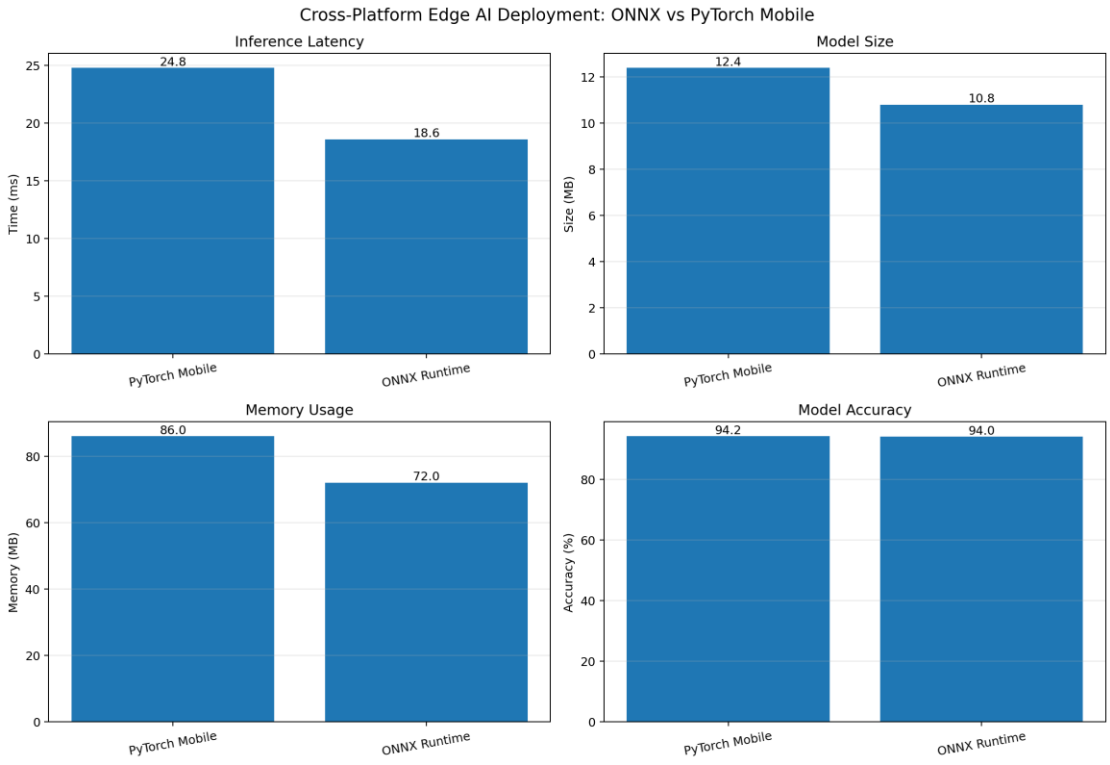


Fig. 2: ONNX Runtime vs PyTorch Mobile (latency, model size, memory usage, accuracy)

Results:

Metric	PyTorch Mobile	ONNX Runtime	Change with ONNX
Inference latency (ms)	24.8	18.6	25.00% lower
Model size (MB)	12.4	10.8	12.90% lower
Memory usage (MB)	86.0	72.0	16.28% lower
Accuracy (%)	94.2	94.0	0.20 points lower

ONNX Runtime was faster (18.6 ms against 24.8 ms), smaller (10.8 MB against 12.4 MB) and used less memory (72.0 MB against 86.0 MB) than PyTorch Mobile, while its accuracy was only 0.2 percentage points lower (94.0% against 94.2%).

Result:

In this comparison ONNX Runtime reduced latency by 25.00%, model size by 12.90% and memory by 16.28% with a negligible accuracy loss of 0.20 percentage points, so it is the better choice for resource-limited edge devices where speed and memory matter, while PyTorch Mobile remains slightly more accurate. The values are fixed simulated figures defined in the code, not measurements from running the model on a device, so they illustrate the expected trade-off rather than real benchmark results.